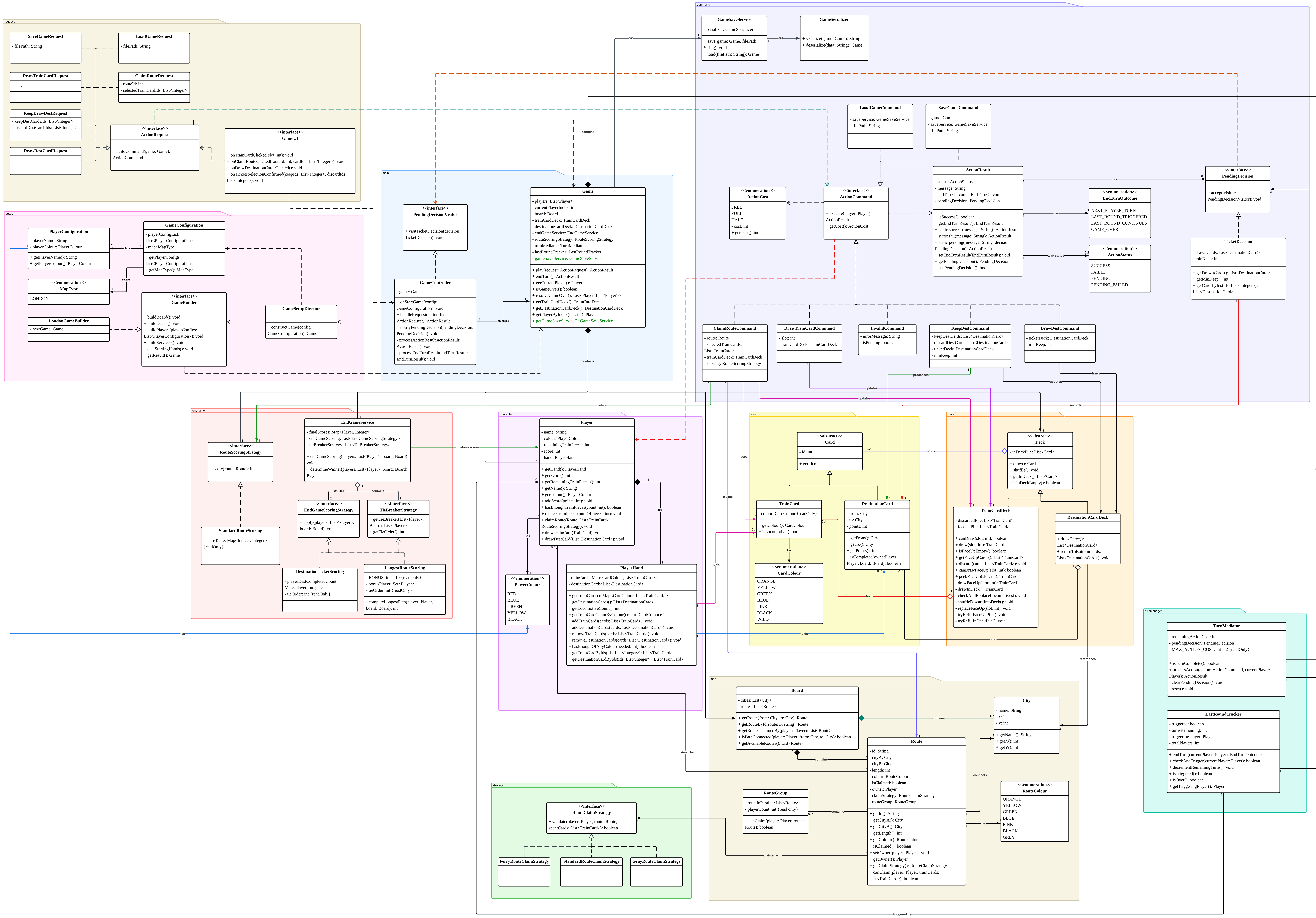
request
SaveGameRequest
- filePath: String
LoadGameRequest
- filePath: String
DrawTrainCardRequest
- slot: int
ClaimRouteRequest
- routeId: int
- selectedTrainCardIds: List<Integer>
KeepDrawDestRequest
- keepDestCardsIds: List<Integer>
- discardDestCardsIds: List<Integer>
DrawDestCardRequest
<<interface>>
ActionRequest
+ buildCommand(game: Game): ActionCommand
<<interface>>
GameUI
+ onTrainCardClicked(slot: int): void
+ onClaimRouteClicked(routeId: int, cardIds: List<Integer>): void
+ onDrawDestinationCardsClicked(): void
+ onTicketsSelectionConfirmed(keepIds: List<Integer>, discardIds: List<Integer>): void
setup
PlayerConfiguration
- playerName: String
- playerColour: PlayerColour
+ getPlayerName(): String
+ getPlayerColour(): PlayerColour
includes
2
1
GameConfiguration
- playerConfigList: List<PlayerConfiguration>
- map: MapType
+ getPlayerConfigs(): List<PlayerConfiguration>
+ getMapType(): MapType
selects
<<enumeration>>
MapType
LONDON
<<interface>>
GameBuilder
+ buildBoard(): void
+ buildDecks(): void
+ buildPlayers(playerConfigs: List<PlayerConfiguration>): void
+ buildServices(): void
+ dealStartingHands(): void
+ getResult(): Game
LondonGameBuilder
- newGame: Game
GameSetupDirector
+ constructGame(config: GameConfiguration): Game
main
<<interface>>
PendingDecisionVisitor
+ visitTicketDecision(decision: TicketDecision): void
GameController
- game: Game
+ onStartGame(config: GameConfiguration): void
+ handleRequest(actionReq: ActionRequest): ActionResult
+ notifyPendingDecision(pendingDecision: PendingDecision): void
- processActionResult(actionResult: ActionResult): void
- processEndTurnResult(endTurnResult: EndTurnResult): void
manages
Game
- players: List<Player>
- currentPlayerIndex: int
- board: Board
- trainCardDeck: TrainCardDeck
- destinationCardDeck: DestinationCardDeck
- endGameService: EndGameService
- routeScoringStrategy: RouteScoringStrategy
- turnMediator: TurnMediator
- lastRoundTracker: LastRoundTracker
- gameSaveService: GameSaveService
+ play(request: ActionRequest): ActionResult
+ endTurn(): ActionResult
+ getCurrentPlayer(): Player
+ isGameOver(): boolean
+ resolveGameOver(): List<Player, List<Player>>
+ getTrainCardDeck(): TrainCardDeck
+ getDestinationCardDeck(): DestinationCardDeck
+ getPlayerByIndex(ind: int): Player
+ getGameSaveService(): GameSaveService
contains
contains
command
GameSaveService
- serializer: GameSerializer
+ save(game: Game, filePath: String): void
+ load(filePath: String): Game
has
GameSerializer
+ serialize(game: Game): String
+ deserialize(data: String): Game
LoadGameCommand
- saveService: GameSaveService
- filePath: String
SaveGameCommand
- game: Game
- saveService: GameSaveService
- filePath: String
<<enumeration>>
ActionCost
FREE
FULL
HALF
- cost: int
+ getCost(): int
<<interface>>
ActionCommand
+ execute(player: Player): ActionResult
+ getCost(): ActionCost
ActionResult
- status: ActionStatus
- message: String
- endTurnOutcome: EndTurnOutcome
- pendingDecision: PendingDecision
+ isSuccess(): boolean
+ getEndTurnResult(): EndTurnResult
+ static success(message: String): ActionResult
+ static fail(message: String): ActionResult
+ static pending(message: String, decision: PendingDecision): ActionResult
+ setEndTurnResult(EndTurnResult): void
+ getPendingDecision(): PendingDecision
+ hasPendingDecision(): boolean
has
<<enumeration>>
EndTurnOutcome
NEXT_PLAYER_TURN
LAST_ROUND_TRIGGERED
LAST_ROUND_CONTINUES
GAME_OVER
with status
<<enumeration>>
ActionStatus
SUCCESS
FAILED
PENDING
PENDING_FAILED
<<interface>>
PendingDecision
+ accept(visitor: PendingDecisionVisitor): void
TicketDecision
- drawnCards: List<DestinationCard>
- minKeep: int
+ getDrawnCards(): List<DestinationCard>
+ getMinKeep(): int
+ getCardsbyIds(ids: List<Integer>): List<DestinationCard>
ClaimRouteCommand
- route: Route
- selectedTrainCards: List<TrainCard>
- trainCardDeck: TrainCardDeck
- scoring: RouteScoringStrategy
DrawTrainCardCommand
- slot: int
- trainCardDeck: TrainCardDeck
InvalidCommand
- errorMessage: String
- isPending: boolean
KeepDestCommand
- keepDestCards: List<DestinationCard>
- discardDestCards: List<DestinationCard>
- ticketDeck: DestinationCardDeck
- minKeep: int
DrawDestCommand
- ticketDeck: DestinationCardDeck
- minKeep: int
processes
updates
updates
updates
draws
records
refers
uses
claims
endgame
<<interface>>
RouteScoringStrategy
+ score(route: Route): int
StandardRouteScoring
- scoreTable: Map<Integer, Integer> {readOnly}
EndGameService
- finalScores: Map<Player, Integer>
- endGameScoring: List<EndGameScoringStrategy>
- tieBreakerStrategy: List<TieBreakerStrategy>
+ endGameScoring(players: List<Player>, board: Board): void
+ determineWinner(players: List<Player>, board: Board): Player
finalizes scores
contains
contains
<<interface>>
EndGameScoringStrategy
+ apply(players: List<Player>, board: Board): void
<<interface>>
TieBreakerStrategy
+ getTieBreaker(List<Player>, Board): List<Player>
+ getTieOrder(): int
DestinationTicketScoring
- playerDestCompletedCount: Map<Player, Integer>
- tieOrder: int {readOnly}
LongestRouteScoring
- BONUS: int = 10 {readOnly}
- bonusPlayer: Set<Player>
- tieOrder: int {readOnly}
- computeLongestPath(player: Player, board: Board): int
has
character
Player
- name: String
- colour: PlayerColour
- remainingTrainPieces: int
- score: int
- hand: PlayerHand
+ getHand(): PlayerHand
+ getScore(): int
+ getRemainingTrainPieces(): int
+ getName(): String
+ getColour(): PlayerColour
+ addScore(points: int): void
+ hasEnoughTrainPieces(count: int): boolean
+ reduceTrainPieces(numOfPieces: int): void
+ claimRoute(Route, List<TrainCard>, RouteScoringStrategy): void
+ drawTrainCard(TrainCard): void
+ drawDestCard(List<DestinationCard>): void
has
has
<<enumeration>>
PlayerColour
RED
BLUE
GREEN
YELLOW
BLACK
PlayerHand
- trainCards: Map<CardColour, List<TrainCard>>
- destinationCards: List<DestinationCard>
+ getTrainCards(): Map<CardColour, List<TrainCard>>
+ getDestinationCards(): List<DestinationCard>
+ getLocomotiveCount(): int
+ getTrainCardCountByColour(colour: CardColour): int
+ addTrainCards(cards: List<TrainCard>): void
+ addDestinationCards(cards: List<DestinationCard>): void
+ removeTrainCards(cards: List<TrainCard>): void
+ removeDestinationCards(cards: List<DestinationCard>): void
+ hasEnoughOfAnyColour(needed: int): boolean
+ getTrainCardByIds(ids: List<Integer>): List<TrainCard>
+ getDestinationCardByIds(ids: List<Integer>): List<TrainCard>
holds
holds
claimed by
card
<<abstract>>
Card
- id: int
+ getId(): int
holds
TrainCard
- colour: CardColour {readOnly}
+ getColour(): CardColour
+ isLocomotive(): boolean
has
<<enumeration>>
CardColour
ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
WILD
DestinationCard
- from: City
- to: City
- points: int
+ getFrom(): City
+ getTo(): City
+ getPoints(): int
+ isCompleted(ownerPlayer: Player, board: Board): boolean
deck
<<abstract>>
Deck
- inDeckPile: List<Card>
+ draw(): Card
+ shuffle(): void
+ getInDeck(): List<Card>
+ isInDeckEmpty(): boolean
TrainCardDeck
- discardedPile: List<TrainCard>
- faceUpPile: List<TrainCard>
+ canDraw(slot: int): boolean
+ draw(slot: int): TrainCard
+ isFaceUpEmpty(): boolean
+ getFaceUpCards(): List<TrainCard>
+ discard(cards: List<TrainCard>): void
+ canDrawFaceUp(slot: int): boolean
+ peekFaceUp(slot: int): TrainCard
- drawFaceUp(slot: int): TrainCard
- drawInDeck(): TrainCard
- checkAndReplaceLocomotives(): void
- shuffleDiscardIntoDeck(): void
- replaceFaceUp(slot: int): void
- tryRefillFaceUpPile(): void
- tryRefillInDeckPile(): void
DestinationCardDeck
+ drawThree(): List<DestinationCard>
+ returnToBottom(cards: List<DestinationCard>): void
holds
holds
references
map
Board
- cities: List<City>
- routes: List<Route>
+ getRoute(from: City, to: City): Route
+ getRouteById(routeID: string): Route
+ getRoutesClaimedBy(player: Player): List<Route>
+ isPathConnected(player: Player, from: City, to: City): boolean
+ getAvailableRoutes(): List<Route>
contains
contains
City
- name: String
- x: int
- y: int
+ getName(): String
+ getX(): int
+ getY(): int
Route
- id: String
- cityA: City
- cityB: City
- length: int
- colour: RouteColour
- isClaimed: boolean
- owner: Player
- claimStrategy: RouteClaimStrategy
- routeGroup: RouteGroup
+ getId(): String
+ getCityA(): City
+ getCityB(): City
+ getLength(): int
+ getColour(): RouteColour
+ isClaimed(): boolean
+ setOwner(player: Player): void
+ getOwner(): Player
+ getClaimStrategy(): RouteClaimStrategy
+ canClaim(player: Player, trainCards: List<TrainCard>): boolean
connects
has
RouteGroup
- routeInParallel: List<Route>
- playerCount: int {read only}
+ canClaim(player: Player, route: Route): boolean
contains
claimed with
<<enumeration>>
RouteColour
ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
GREY
strategy
<<interface>>
RouteClaimStrategy
+ validate(player: Player, route: Route, spentCards: List<TrainCard>): boolean
FerryRouteClaimStrategy
StandardRouteClaimStrategy
GrayRouteClaimStrategy
turnmanager
TurnMediator
- remainingActionCost: int
- pendingDecision: PendingDecision
- MAX_ACTION_COST: int = 2 {readOnly}
+ isTurnComplete(): boolean
+ processAction(action: ActionCommand, currentPlayer: Player): ActionResult
- clearPendingDecision(): void
- reset(): void
tracks
LastRoundTracker
- triggered: boolean
- turnsRemaining: int
- triggeringPlayer: Player
- totalPlayers: int
+ endTurn(currentPlayer: Player): EndTurnOutcome
+ checkAndTrigger(currentPlayer: Player): boolean
+ decrementRemainingTurns(): void
+ isTriggered(): boolean
+ isOver(): boolean
+ getTriggeringPlayer(): Player
triggered by